# INTERNSHIPREPORT

ON

# "JAVA FULL STACK WEB DEVELOPMENT"

AT

**INSTITUTE FOR SYSTEMS STUDIES & ANALYSES (ISSA)**
**DEFENCERESEARCH&DEVELOPMENTORGANISATION(DRDO)**
**METCALFE HOUSE, DELHI - 110054**

UNDERTHEGUIDANCEOF

**SHUBHI SAINI**

**(SCIENTIST'E',ISSA)**

SUBMITTEDBY

**ANKIT RAJ**

**BACHELORSOFTECHNOLOGY**

**IN**

**ARTIFICIALINTELLIGENCEANDDATASCIENCE**
**ARTIFICIALINTELLIGENCEANDDATASCIENCE DELHI**
**TECHNICAL CAMPUS**
**(AffiliatedtoGuruGobindSinghIndraprasthaUniversity—GGSIPU)**

# DECLARATION

I, Ankit Raj, Roll No. 27180119222 student of Bachelor of Technology in Artificial Intelligence and Data Science at Delhi Technical Campus, affiliated to Guru Gobind Singh Indraprastha University (GGSIPU), Delhi, hereby declare that this Internship Report titled "Java Full stack Web Development" is being submitted in partial fulfilment of the requirements of the training programme undertaken at the Institute for Systems Studies & Analyses (ISSA), Defence Research and Development Organisation (DRDO), Metcalfe House, Delhi - 110054.

This report is an authentic and original record of the work performed by me during the internship period under thesupervisionand guidance of my assigned mentor at ISSA. All technical work, analysis, and findings documented herein are a result of my own efforts and learning during the training.

I, hereby declare that this report is being submitted in fulfilment of the **Trainin Programme in Institute for Systems Studies & Analyses (ISSA), Defence Research and Development Organisation** and is result of work carried out by me under the guidance of **Mrs.SHUBHI SAINI (SC 'E')**.

I further declare that to my knowledge, the structure and content of this report are original and have not been submitted before any purpose.

**AnkitRaj**
B.Tech—ArtificialIntelligenceandDataScience Roll

No:02718011922

DelhiTechnicalCampus,GGSIPU

Date: _____

# CERTIFICATE

This is to certify that Mr. Ankit Raj, student of Bachelor of Technology in Artificial Intelligence and Data Science from Delhi Technical Campus, affiliated to Guru Gobind Singh Indraprastha University (GGSIPU), Delhi, has successfully completed the internship training programme at the Institute for Systems Studies & Analyses (ISSA), Defence Research and Development Organisation (DRDO), Metcalfe House, Delhi — 110054.

During the internship, the student worked on the development of a web-based data visualization tool using Java Full Stack technologies including Java, JSP (JavaServer Pages), HTML, CSS, and JavaScript. The student demonstrated sincere dedication, technical aptitude, and professional conduct throughout the training period.

The work documented in this report is a genuine account of the contributions made by the student during the training programme. This certificate is issued in recognition of the successful completion of the internship.

**SHUBHI SAINI**
Scientist,'E',ISSA

DefenceResearchandDevelopmentOrganisation(DRDO)

Metcalfe House, Delhi - 110054

Date: _____

## ACKNOWLEDGEMENT

Training is one of the most vital components of an engineering curriculum. It serves as the bridge between academicknowledgeand professionalpractice, offeringstudents the opportunity to experience the realities of the working world — the complexities, the disciplines, the collaborations, and the rewards that come with building something that truly matters. It is in this spirit that the present internship, undertaken at one of India's most prestigious defense research institutions, has been an experience ofimmeasurable value.

It gives me immense pleasure and a deep sense of fulfillment to have successfully completed my summer internship at the Institute for Systems Studies and Analyses (ISSA), Defence Research and Development Organisation (DRDO), Delhi, and topresent this report as a comprehensive record of the work and learning that took place during this period.

I am wholeheartedly thankful to my guide and mentor at ISSA for their invaluable guidance, consistent encouragement, and patient mentorship throughout the internship. Their deep technical knowledge, their willingness to share insights and explain complex concepts, and their constructive feedback on my work were instrumental in the successful completion of my assigned tasks. I could not have asked for a more supportive and inspiring mentor.

I would also like to thank, **Mr.Sunil yadav , Technical Officer 'A', ISSA**
for his invaluable guidance and suggestions during the development of the projects.

I owe a debt of gratitude to my parents and family, whose unwavering support, encouragement, and belief in my abilities have been my greatest source of strength and motivation throughout my academic journey.



**AnkitRaj**
B.TechFinalYear—ArtificialIntelligenceandDataScience

Delhi Technical Campus, GGSIPU

# ABSTRACT

This report presents a detailed account of the internship training carried out at the Institute for Systems Studies and Analyses (ISSA),Defence Research andDevelopment Organisation (DRDO), Delhi, during the summer training programme. The internship focused on the design and development of a web-based data visualizationtool using Java Full Stack technologies.

The primary objective of the internship was to gain hands-on exposure to modern web application development using server-side Java technologies, specifically JavaServer Pages (JSP) integrated with a Java backend, HTML, CSS, and JavaScript on the frontend, and PostgreSQL as the relational database backend. The project involved building an interactive, browser-based visualization interface capable of dynamically rendering real-time data received from backend services.

The backend of the tool is built using Java Servlets and JSP, which process data from the PostgreSQL database and serve it to the frontend. RESTful API endpoints facilitate structured data exchange between the frontend and backend. JavaScript on thefrontend handles dynamic rendering, state management, and user interaction.

# TABLEOFCONTENTS

1. Introduction

1.1 Background

1.2 AbouttheOrganization

1.3 ScopeoftheInternship

2. Objectives

2.1 PrimaryObjective

2.2 SecondaryObjectives

2.3 ExpectedOutcomes

3. TechnologyStack

3.1 Overview

3.2 Java

3.3 JavaServerPages(JSP)

3.4 HTML

3.5 CSS3

3.6 JavaScript

3.7 PostgreSQL

3.8 Git—VersionControl

4. Methodology

4.1 Phase1—RequirementAnalysisandDesign

4.2 Phase2—Development

4.3 Phase3—TestingandQualityAssurance

4.4 Phase4—IntegrationandReview

5. SystemArchitectureandDesign

5.1 ArchitecturalOverview

5.2 FrontendArchitecture

5.3 BackendArchitecture

5.4 DatabaseDesign

5.5 CommunicationFlow

6. VisualizationTool—DesignandDevelopment

6.1 ToolOverview

6.2 DisplayPanelArchitecture

6.3 InformationSummaryPanel

6.4 DynamicDataRendering

6.5 LayerandColorManagement

6.6 BackendDataIntegration

7. SupportingModulesDeveloped

7.1 DataConfigurationModule

7.2 EntityValidationModule

7.3 SearchandFilterModule

7.4 Instructor/AdminControlModule

8. ChallengesandSolutions

9. Learnings

9.1 TechnicalSkillsGained

9.2 ProfessionalSkillsGained

10. Conclusion

10.1 Summary

10.2 FutureScope

11. BibliographyandReferences

# 1. INTRODUCTION

## 1.1 Background

In the contemporary landscape of software engineering, the demand for interactive,data-driven web applications has grown exponentially across all sectors — from commercial businesses to healthcare, scientific research, and national defense. Organizations that deal with large volumes of real-time or near-real-time data increasingly require sophisticated visualization tools that can translate complexnumerical and spatial data into clear, intuitive, and actionable graphical representations.

Forresearchanddevelopmentorganizations inparticular,suchvisualizationtoolsplay a critical role in the analysis, simulation, and decision-making processes that underpin their core activities. The ability to visually represent the output of complex computational models, display the status of multiple interrelated systems simultaneously, and provide operators with an at-a-glance understanding of a dynamic situation is not merely a convenience — it is a fundamental operational requirement.

This internship was undertaken in the context of this growing need for advanced visualization capabilities. The project assigned during the training programme involved the design and full-stack development of a web-based visualization tool using Java technologies, with a focus on dynamic rendering, real-time data integration, and a clean, professional user interface.

## 1.2 AbouttheOrganization

The Defence Research and Development Organisation (DRDO) is India's premier government agency responsible for the research, design, development, and production of advanced technology for military use. Established in 1958 and operating under the Ministry of Defence, Government of India, DRDO comprises a network of more than 50 specialized laboratories spread across the country, each focusing on a specific domainof defense technology.

The Institute for Systems Studies and Analyses (ISSA), located at Metcalfe House,Delhi, is one of DRDO's key research establishments. ISSA's primary mandate encompasses systems analysis, operational research, mathematical modeling, simulation, and the development of decision-support software systems for the Indian Armed Forces. The institute regularly develops advanced analytical tools and software platforms that assist military commanders and planners in making informed, data-driven decisions.

ISSA follows stringent software development standards and quality assurance practices commensurate with the mission-critical nature of its applications. Working within this environment provided an excellent foundation for understanding professional software development at the highest level of rigor and accountability.

## 1.3 ScopeoftheInternship

TheinternshipatISSA,DRDOspannedseveralweeksandencompassedthefollowing broad areas of work:

- Designanddevelopmentofaweb-basedvisualizationtoolforreal-timespatial and numerical data display.
- Developmentofsupportingbackendmodulesfordatamanagement,validation, configuration, and control.
- IntegrationoffrontendcomponentswithJava-basedbackendservicesanda PostgreSQL relational database.
- Testing,debugging,andqualityassuranceofthedevelopedsoftware.
- Documentationofthedesigndecisions,implementationdetails,andoutcomes of the project.

# 2. OBJECTIVES

## 2.1 PrimaryObjective

The primary objective of the internship was to design, develop, and integrate a fully functionalweb-based datavisualizationtool usingJavaFullStacktechnologies.The tool was required to dynamically display spatial and numerical data in a visually intuitive format, update in real time as new data became available, and provide users with a comprehensive informational summary panel alongside the main visual display.

The toolneededtoberobust,responsive, andcapable of handling varying data volumes without performance degradation. It was also required to integrate seamlessly with the existing backend infrastructure, fetching data through well-defined API endpoints and rendering it on a dynamic HTML canvas interface.

## 2.2 SecondaryObjectives

Inadditiontotheprimaryvisualizationtool,severalsecondaryobjectivesweredefined for the internship:

- Todevelopsupportingbackendmodulesincludingdataconfigurationpanels, entity management forms, and administrative control interfaces.
- Toimplementcomprehensivevalidationlogicatboththefrontendandbackend levels to ensure data correctness and system reliability.
- Todiagnoseandresolvedatapersistenceissuesinexistingmoduleswhere database updates were not being correctly executed.
- Toimplementoverlapdetectionandconstraintenforcementforspatialdata entry modules.
- TogainpracticalproficiencyinJavaServlets,JSP,PostgreSQL,andGitwithin a professional team environment.
- Todevelopathoroughunderstandingofthesoftwaredevelopmentlifecycleas practiced in a defense research organization.

## 2.3 ExpectedOutcomes

Attheconclusionoftheinternship,thefollowingoutcomeswereexpected:

1. Afullyfunctional,integratedweb-basedvisualizationtoolaccessiblethrougha standard web browser.
2. Enhancedandcorrectedbackendservicemoduleswithresolveddata persistence and validation issues.
3. Acomprehensiveinternshipreportdocumentingthedesign,development,and learning outcomes of the training programme.
4. PracticalproficiencyinJavaFullStackdevelopment,PostgreSQLdatabase management, and professional software engineering practices.

# 3. TECHNOLOGYSTACK

## 3.1 Overview

The technology stack selected for the internship project reflects both the existing infrastructure at ISSA and the most appropriate tools for the requirements of the visualization tool. The stack is centered on Java as the primary backend language, with JSP(JavaServerPages) serving as the server-sidetemplating technology for generating dynamic HTML content. The frontend is built using standard webtechnologies — HTML, CSS3, and JavaScript — and the data persistence layer uses PostgreSQL.

Thefollowingtableprovidesaconcisesummaryofthetechnologystack:

| Technology | Category | PrimaryRole |
|---|---|---|
| Java(JDK8+) | Backend | Corebusinesslogic, Servlets, APIs |
| JSP(JavaServerPages) | Server-Side | Dynamic HTML generation, view layer |
| HTML | Frontend | Structure and layout of web pages |
| CSS | Frontend | Styling,layout, responsiveness |
| JavaScript | Frontend | Dynamic behavior, data rendering, API calls |
| PostgreSQL | Database | Relational data storage and management |
| JDBC | DBConnectivity | Java-to-PostgreSQL connection layer |
| Git | VersionControl | Sourcecodemanagement |
| ApacheTomcat | WebServer | Servletcontainer/ applicationserver |

## 3.2 Java

Java isthecoretechnology poweringthebackendofthevisualizationtool.Asoneof the most mature, widely adopted, and thoroughly tested programming languages in the world, Java offers a combination of performance, reliability, and a rich ecosystem of libraries and frameworks that make it the language of choice for enterprise and government applications.

- JavaServlets:Usedastheprimaryserver-sidecomponentforhandlingHTTP requests and generating dynamic responses.
- Object-OrientedDesign:Alldataentities(visualizationdatapoints,configuration parameters, user inputs) were modeled as Java classes with well-defined attributes and methods.
- ExceptionHandling:Comprehensivetry-catchblocksensuregracefulerror handling and meaningful error messages throughout the application.
- CollectionsandStreams:JavaCollectionsandtheStreamAPIwereusedfor efficient processing, filtering, and transformation of data sets retrieved from PostgreSQL.
- JDBC (Java Database Connectivity): JDBC was used for all database interactions,includingexecutingSQLqueries,managingtransactions,and processing result sets.

### 3.3 JavaServer Pages(JSP)

JavaServer Pages (JSP) is a server-side web technology that enables the creation of dynamic web content by embedding Java code directly within HTML pages. JSP pages are compiled into Java Servlets by the application server (Apache Tomcat) andexecuted on theserver to generate HTML thatissent to the client's browser.

In this project, JSP was used as the view layer of the application, responsible for generating the initial HTML structure of the visualization tool's web interface. JSP's ExpressionLanguage(EL)andJSTL(JSPStandardTagLibrary)wereusedto

dynamically injectserver-side data intothe HTML output, enabling theinitialrendering of configuration data and page statewithoutrequiring additional client-sideAPI calls.

### 3.4 HTML

HTML (HyperText Markup Language, version 5) is the foundational markup language used to define the structure and content of all web pages in the visualization tool. HTML introduces several powerful features beyond its predecessors that were directly relevant to this project, most notably theCanvas element and improved semantic markup.

- Canvas Element: The HTML Canvas API was used to render the dynamic visualization display, including the sector-shaped spatial display and the animateddatapointmarkers.Canvasprovidesapixel-leveldrawingAPIthatis ideal for custom, data-driven graphical outputs.
- SemanticElements:HTMLsemanticelementssuchasheader,main,section, article, and footer were used throughout the application to improve code readability, accessibility, and SEO compatibility.
- Form Elements: HTML form elements with built-in validation attributes were usedfordataentryinterfaces,reducingtheneedforcustomvalidationcodein simple cases.
- DataAttributes:HTMLdataattributeswereusedtoembedmetadataintoDOM elements, facilitating clean communication between the HTML structure and JavaScript logic.

### 3.5 CSS

CSS (Cascading Style Sheets, version ) was used to define the visual presentation of the entire web application — controlling layout, typography, color schemes, spacing, animations, and responsiveness across different screen sizes.

- Flexbox and CSS Grid: These modern layout systems were used to create structured,responsivepagelayoutsthatadaptgracefullytodifferentscreen

resolutions,particularlyimportantforthedual-panellayoutofthevisualization tool (main display panel + information summary panel).

- CSSTransitionsandAnimations:Subtleanimationswereappliedtodatapoint markers and panel updates to improve the user experience and make state changes visually apparent.
- Custom Properties (CSS Variables): CSS custom properties were used to defineaconsistentcolorpaletteandspacingsystem,makingthevisualdesign easy to adjust and maintain.
- MediaQueries:Responsivedesignmediaqueriesensurethetool'sinterface remains usable across desktop workstations, laptops, and tablets.

### 3.6 JavaScript

JavaScriptistheprogramminglanguagethatpowersalldynamic,client-sidebehavior in the visualization tool. It is responsible for fetching data from the backend APIs, processingthereceiveddata, anddynamically rendering it onthe HTMLCanvasdisplay. JavaScript also manages all user interaction events, controls the state of the UI, and updates the information summary panel in real time.

- Async/Await and Fetch API: Used for asynchronous HTTP requests to the backendRESTAPIs,ensuringtheUIremainsresponsivewhiledataisbeing fetched from the server.
- ArrowFunctionsandDestructuring:ES6+syntaxfeaturesthatimprovecode conciseness and readability.
- DOMManipulation:TheDocumentObjectModelAPIwasusedtodynamically create, modify, and remove HTML elements in response to data changes and user actions.
- Canvas 2D Context API: The Canvas 2D drawing API was used to programmaticallydrawthevisualizationdisplay,includingthesectorshape, range rings, angular grid lines, and data point markers.
- Event Listeners: Event-driven programming was used to respond to user interactionssuchasbuttonclicks,formsubmissions,andhovereventsonthe visualization canvas.

- JSON Processing: All data exchanged with the backend is in JSON format; JavaScript'snativeJSON.parse()andJSON.stringify()methodswereusedfor serialization and deserialization.

### 3.7 PostgreSQL

PostgreSQLisapowerful,open-source,object-relationaldatabasemanagement system (ORDBMS) renowned for its reliability, feature richness, ACID compliance, and standardsconformance.Itisoneofthemostwidelydeployedopen-sourcedatabases in enterprise and government environments globally.

In this project, PostgreSQL serves as the primary data store for all application data including visualization data records, configuration parameters, entity definitions, and system state information. The database is accessed from the Java backend through JDBC connections managed by a connection pool for efficiency.

- ACID Transactions: All multi-step database operations (such as saving a completeconfiguration withmultiplerelatedentities)arewrappedin transactionstoensureatomicity.Ifanystepfails,theentiretransactionisrolled back to prevent partial saves.
- Relational Schema Design: The database schema follows normalization principles(3NF)tominimizeredundancyandmaintaindataintegrityacross related tables.
- CheckConstraints:Database-levelcheckconstraintsenforcebusinessrules (such as value range limits) independently of application logic, providing a robust last line of defense.
- Foreign Key Constraints: Referential integrity between related tables is enforcedthroughforeignkeys,preventingorphanrecordsandmaintaining consistency.
- IndexedQueries:Indexesonfrequentlyqueriedcolumnsensurefastdata retrieval even as data volumes grow.

### 3.8 Git—VersionControl

Git is the distributed version control system used throughout the project to manage the source code of both the frontend and backend components. In a professional software development environment, version control is not merely a best practice — it is an absolute requirement for maintaining code stability, enabling collaboration, andproviding an audit trail of all changes.

During the internship, Git was used to maintain a clean and organized development history. Each feature addition or bug fix was committed with a descriptive commit message clearly explaining the nature of the change. Feature branches were used for significant new developments, and changes were integrated into the main branch after review and testing. This workflow is consistent with professional software development practices used in industry and government organizations worldwide.

# 4. METHODOLOGY

The development of the visualization tool followed a structured Software Development Life Cycle (SDLC) methodology, adapted to the agile and iterative nature of the work environment at ISSA. The methodology comprised four principal phases: Requirement Analysis and Design, Development, Testing and Quality Assurance, and Integration and Review.Eachphase hadclearly defined activities, deliverables, and success criteria.

## 4.1 Phase 1—RequirementAnalysisand Design

The first phase of the project involved a thorough understanding of the requirements for the visualization tool and the supporting backend modules. This phase included the following activities:

### 4.1.1 RequirementGathering

Requirements were gathered through direct discussions with the assigned mentor and senior scientists at ISSA. The discussions covered the functional requirements of the visualization tool (what data should be displayed, in what format, with what level of detail), the non-functional requirements (performance, responsiveness, security), andthe integration requirements (how the tool should connect to existing backend services and the PostgreSQL database).

### 4.1.2 SystemDesign

Based on the gathered requirements, a system design was developed that defined the overall architecture of the tool, the structure of the frontend interface, the design of backend API endpoints, and the database schema for storing visualization data. The design followed the MVC (Model-View-Controller) pattern, with a clear separation between the data model (Java classes and PostgreSQL tables), the view (JSP/HTML/CSS), and the controller (Java Servlets).

### 4.1.3 UIWireframing

Low-fidelity wireframes were sketched for the visualization tool's interface, defining the placement of the main display panel, the information summary panel, control buttons, and dataindicators. These wireframes werereviewed with thementor and refined based on feedback before development commenced.

## 4.2 Phase2—Development

The development phase was the most extensive phase of the project, spanning the implementation of all frontend and backend components. Development proceeded in a module-by-module manner, beginning with the core visualization display and progressively adding the backend data integration, supporting modules, and validation logic.

### 4.2.1 FrontendDevelopment

### 4.2.2 BackendDevelopment

### 4.2.3 DatabaseImplementation

The PostgreSQL database schema was implemented based on the design created in Phase 1. Tables were created with appropriate data types, primary keys, foreign key constraints, and check constraints. SQL queries for all CRUD operations were written and optimized. Database transactions were configured to ensure atomicity for all multi -step operations.

## 4.3 Phase3—Testing andQualityAssurance

Testing was carried out at multiple levels to ensure the correctness, reliability, and performance of the developed software. Unit testing of individual Java methods was performed to validate logic in isolation. Integration testing was performed to verify the correct end-to-end behavior of each API endpoint, from the HTTP request through the service layer to the database and back. Manual user interface testing was performed in the browser to verify visual correctness and functional behavior under various scenarios including edge cases and error conditions.

Postman was used extensively during testing to send HTTP requests with various payloads to the backend endpoints, enabling precise testing of validation logic, error handling, and data persistence. PostgreSQL's query console was used to directly verify database state after operations, confirming that data was being correctly saved,updated, and retrieved.

### 4.4 Phase4— IntegrationandReview

The final phase involved the integration of all developed components into a cohesive, working application deployed on the Apache Tomcat application server. Integration testing was performed on the complete system to identify any interface or data compatibility issues between modules. The completed work was reviewed with the mentor and senior scientists, feedback was incorporated, and final refinements were made to both the code and the documentation.

# 5. SYSTEMARCHITECTUREANDDESIGN

## 5.1 Architectural Overview

The visualization tool is built on a three-tier web application architecture that separates the concerns of the presentation layer, the application logic layer, and the data persistence layer. This architectural approach is a widely adopted standard forenterprise web applications and is particularly suitable for the requirements of thisproject due to its modularity, maintainability, and scalability.

- Presentation Tier (Frontend): HTML, CSS3, and JavaScript components renderedintheuser'swebbrowser.Responsibleforallvisualdisplayanduser interaction.
- ApplicationLogicTier(Backend):JavaServletsandJSPrunningonApache Tomcat. Responsible for processing requests, applying business logic, performing validations, and managing data flow.
- DataTier(Database):PostgreSQLrelationaldatabase.Responsiblefor persistent, structured storage of all application data.

## 5.2 FrontendArchitecture

The frontend is organized into three functional zones. The Main Display Panel occupies the primary portion of the screen and hosts the HTML Canvas-based visualization. The Information Panel is positioned alongside the main display and presents a tabular summary of the data being visualized. The Control Bar at the top of the page provides navigational controls, display mode toggles, and action buttons for data management operations.

The frontend JavaScript codebase is organized into modules, each responsible for a specific concern: the rendering module handles all Canvas drawing operations, the data module manages API communication and data processing, and the UI module handles all DOM manipulation and event handling. This modular organization improves code maintainability and testability.

## 5.3 BackendArchitecture

The backend follows the MVC (Model-View-Controller) pattern. Java Servlet controllers receive HTTP requests, extract and validate request parameters, and delegate processing to service-layer Java classes. Service classes contain all business logic and interact with the database throughJDBC-based data access objects(DAOs). JSP pages serve as the view layer, rendering dynamic HTML content on the server side.

## 5.4 DatabaseDesign

The PostgreSQL database schema is designed to support efficient storage and retrieval of all data required by the visualization tool and its supporting modules. The schema follows third normal form (3NF) to minimize redundancy. Key design principles include the use of UUID primary keys for all tables, foreign key constraints to enforce referential integrity, and check constraints to enforce business rules at the database level. Indexes areapplied to all frequently queriedcolumnsincluding foreign keys and status fields.

## 5.5 CommunicationFlow

The communication flow between the frontend and backend follows a standard RESTful request-response pattern. When the user loads the visualization tool page, JSP generates the initial HTML. JavaScript then asynchronously fetches real-time data from the backend via REST API calls. The backend retrieves data from PostgreSQL, processes it, and returns a JSON response. JavaScript processes the JSON and updates the Canvas display and information panel. This cycle repeats at definedintervals to keep the display current. All error conditions are handled gracefully, with appropriate messages displayed to the user.

# 6. VISUALIZATIONTOOL—DESIGNANDDEVELOPMENT

## 6.1 ToolOverview

The web-based visualization tool developed during the internship is designed to display dynamic spatial and numerical data in an interactive, browser-based interface. The tool receives data from backend Java services (which in turn retrieve it from the PostgreSQL database) and renders it in real time on a two-component interface: a main graphical display panel and an accompanying information summary panel.

The tool is intended for use in an operational control environment where operators need to monitor the position, bearing,and range of multiple data entities simultaneously andin real time. The design prioritizes clarity, precision, and responsiveness, using a high-contrast dark color scheme that is easy to read under varied lighting conditions.

## 6.2 DisplayPanelArchitecture

The main display panel is rendered on an HTML Canvas element and depicts a sector-shapedspatialvisualization.Thedesignofthispanelwasguidedbytherequirements to display entities at varying angular positions and distances within a defined coverage arc.

### 6.2.1 SectorShapeandRangeRings

The display sector represents a forward-facing arc, centered at the bottom of thecanvas. Concentric arc segments (rangerings) are drawn atregulardistance intervals
— at 200 units, 400 units, 600 units, and 800 units from the center origin. These rings provide a visual reference for estimating the distance of displayed entities. Distance labels are rendered at each ring level on both the left and right sides of the display for easy reading.

### 6.2.2 AngularGridLines

A central vertical axis line divides the sector symmetrically and serves as the zero-degree reference line. This line, along with the arc segments, creates a grid that allows operators to quickly estimate the angular position of any data point within the display. The zero-degree label is rendered at the top of the central axis.

### 6.2.3 DataPointMarkers

Individual data entities are displayed as circular markers positioned within the sector according to their angular position (bearing) and distance (range) from the center. The markers are rendered with a distinct color and a subtle glow effect to make them clearly visible against the dark background of the display sector. When a user hovers over a marker, a tooltip is displayed showing the entity's label, angular position, and distance value.

### 6.2.4 CoordinateMapping

## 6.3 InformationSummaryPanel

Adjacent to the main display canvas, an information summary panel presents a tabular view of all currently displayed data entities. Each row in the table corresponds to one data entity and displays its identifier, angular position, and range distance. The panel is styled with a dark background and high-contrast text to match the aesthetic of the main display panel.

The information panel is dynamically updated whenever the Canvas display is refreshed with new data. JavaScript synchronizes the panel content with the Canvas display to ensure they always show consistent information. The panel uses a fixed-heightcontainer with vertical scrolling to accommodate large numbers of data entries without distorting the page layout.

## 6.4 DynamicDataRendering

The visualization tool is designed to operate in a near-real-time data display mode. JavaScript periodically sends asynchronous GET requests to the backend REST API endpoint dedicated to serving visualization data. The backend retrieves the latest data records from the PostgreSQL database and returns them as a JSON array. JavaScript processes this array, clears the previous Canvas content, and redraws all data point markers at their updated positions.

## 6.5 LayerandColorManagement

The visualization display is organized into multiple visual layers rendered in a defined drawing order on the Canvas. The background layer establishes the dark base of the display. The sector fill layer renders a semi-transparent color fill within the display arc, providing depth and contrast. The grid layer renders the range rings and angular reference lines. The overlay layers render transparent color fills for designated zones within the display (such as exclusion zones or regions of interest), using transparent RGBA color values that allow the underlying layers to remain visible. Finally, the data layer renders all entity markers on top of all other layers.

## 6.6 BackendData Integration

The backend integration for the visualization tool involves a dedicated Java Servlet that handles GET requests from the frontend JavaScript. Upon receiving a request, the Servlet invokes a service method that executes a parameterized SQL query against the PostgreSQL database to retrieve the current set of data records. The results aremapped to Java objects, serialized to a JSON string using a JSON library, and returned in the HTTP response with the appropriate Content-Type header.

Error handling is implemented at both the Servlet level (catching and logging all exceptions) and the JavaScript level (catching fetch errors and displaying user-friendly messages). This ensures that temporary data retrieval failures, such as brief network interruptions or database query timeouts, are handled gracefully without crashing the visualization display.

# 7. SUPPORTINGMODULESDEVELOPED

## 7.1 DataConfigurationModule

In addition to the main visualization tool, a data configuration module was developed to allow authorized users to define and manage the entities whose data is displayed in the visualization. This module provides a web-based form interface for creating, viewing, editing, and deleting entity configuration records in the PostgreSQL database.

The module features comprehensive form validation implemented at two levels: client-sidevalidationusingJavaScriptprovidesimmediatefeedback tothe userforincorrect or missing inputs without requiring a server round-trip; server-side validation in the Java Servlet provides authoritative validation before any database operation is performed. Validation rules include checks for required fields, numeric range constraints, data type correctness, and business-logic rules specific to entity types.

A significant portion of the work on this module involved fixing data persistence issues where certain field updates were not being correctly savedto the PostgreSQL database. Investigation revealed that the updatelogic inthe Servlet was conditionally executing the SQL UPDATE query only when certain primary fields changed, inadvertentlyskipping updates when only secondary fields were modified. The fix required rewritingthe update logic to perform a complete data save whenever any field in the entity record is changed.

## 7.2 Entity ValidationModule

Adedicatedentityvalidationmodulewasimplementedtoenforcedataintegrityrules across multiple configuration types. Key validation rules implemented include:

- Count Constraints: For entitiesthatcanbeconfiguredwithaquantityparameter (such as a count of sub-components), a maximum allowed value of 150 was enforced,alongwithaminimumvalueof1.Bothconstraintsareimplementedat the Java service layer and reinforced through PostgreSQL check constraints.

- Speed and Rate Constraints: For entities with a speed or rate parameter, validation ensures the value does not exceed the maximum physically realistic valuefortheentitytype.Themaximumallowedvaluesareconfigurablethrough system parameters stored in the database.
- MandatoryFieldValidation:Allrequiredfieldsarevalidatedtoensuretheyare not null or empty before any database write operation is attempted.

This layered approach to validation — frontend JavaScript for immediate feedback, Java service layer for authoritative enforcement, and PostgreSQL check constraints as the final safety net — ensures that invalid data can never reach the database regardless of how a request is submitted.

### 7.3 SearchandFilterModule

A spatial search and filter module was developed to manage the definition of designated search zones within the system's operational data. This module allows authorized users to define circular geographical zones by specifying a center point and a radius.

A key feature of this module is overlap detection: before saving a new zone definition to the database, thesystem checks whether the new zone overlaps with any existing zone. The overlap check is performed using a geometric distance calculation — if the distance betweenthecenter pointsof thenewzoneandanyexistingzoneislessthanthesum of their radii, an overlap is detected.

When an overlap is detected, the system displays a clear error message to the user and does not update the zone's position on the map display. Critically, the zone's previous position in the PostgreSQL database is also not updated — a correction made duringthe internship, as the original implementation was erroneously writing the overlapping positionto the databasedespite the error beingshown on screen.Thefix implemented a conditional database write that executes only when all validations pass.

## 7.4 Instructor/ AdminControlModule

An administrative control module was developed to provide privileged users with enhanced management capabilities over the system's operational data andconfiguration. This module includes features for editing existing saved configurations (which are normally read-only for standard users), controlling the availability of certain data managementfunctionsbasedon thecurrentsystemstate,andgeneratingaudit log entries for administrative actions.

A significant implementation challenge in this module was correctly managing the enabled or disabled state of multiple UI controls based on the current system state. Controls such as the data save button, the configuration lock/unlock toggle, and thereset button each needed to be conditionally enabled or disabled in a coordinated and consistent manner. A centralized state management approach was implemented, where a single JavaScript state object tracks thecurrent system state and is usedas the single source of truth for all control visibility and availability decisions.

# 8. CHALLENGESANDSOLUTIONS

## 8.1 DataPersistenceBuginUpdate Operations

Challenge: One of the most technically demanding challenges encountered during the internship was a recurring data persistence bug that manifested in multiple modules. In each affected module, when a user modified certain fields of an existing record and saved the changes, the UI would confirm the save, but the PostgreSQL database would notreflect theupdatedvalues.Thisledtoaconfusingsituation wheretheUIshowed the new values but a page refresh would revert the display to the old values from the database.

Root Cause: Detailed investigation of the Java Servlet and service layer code revealed that the SQL UPDATE query was being executed conditionally — only when a specific subset of fields had changed. When the modified fields were not in this conditional set (for example,when only acount value changed but the locationremained the same), the condition evaluated to false and the UPDATE query was never executed.

Solution:The fix involvedrewriting theconditional logic in all affectedservice methods to use an unconditional update approach: whenever a save request is received, the service always executes a full SQL UPDATE for the affectedrecord, regardless of which specific fields have changed. This simple but effective fix completely resolved the data persistence issues across all affected modules.

## 8.2 OverlapDetection with DatabaseIntegrity

Challenge: Implementing the overlap detection feature for the search zone module required the system to correctly reject overlapping zone placements at both the UI level (showing an error and reverting the map marker) and the database level (not writing the invalid position to PostgreSQL). The original implementation failed to prevent the database write on overlap, leading to data inconsistency.

Solution: The service method was restructured with an explicit early-return pattern: the overlap detection check is performed first. If an overlap is detected, the method immediately returns an error response to the frontend without executing any database writeoperation.Thefrontendcatchesthiserrorresponseandrevertsthemapmarker to its previous position. This clean separation of the validation check and the database write ensures that the two layers remain perfectly synchronized.

# 9. LEARNINGS

## 9.1 TechnicalSkillsGained

The internship provided extensive practical exposure to a wide range of technical skills that significantly augmented the theoretical knowledge acquired during the academic programme.

### 9.1.1 JavaFullStackDevelopment

The most substantial technical learning of the internship was the development ofgenuineproficiency in Java Full Stackweb development. This encompasses not only the ability to write Java code, but a comprehensive understanding of how all the components of a Java web application — Servlets, JSP, service layers, JDBC, and deployment on Apache Tomcat — fit together and interact to produce a functioning application. Prior to the internship, knowledge of these technologies was largely theoretical. The internship transformed this knowledge into practical, hands-on competence.

### 9.1.3 PostgreSQLDatabaseManagement

PracticalproficiencyinPostgreSQLwasoneofthemostvaluabletechnicaloutcomes of the internship. Skills developed include: designing and implementing relational database schemas; writing efficient SQL queries including joins, aggregations, and parameterized queries; implementing ACID transactions for multi-step operations; configuring database constraints (primary keys, foreign keys, check constraints, unique constraints); and diagnosing and resolving data integrity issues. These are foundational database skills that are directly applicable tovirtually anysoftware development role.

### 9.1.4 HTMLCanvasAPI

The development of the visualization tool's display panel provided intensive hands-on experience with the HTML Canvas 2D drawing API. Skills developed include drawing geometric shapes (arcs, lines, circles), applying transformations (rotations, translations,

scaling), managing multiple drawing layers, implementing animations, and performing coordinate system transformations. Canvas-based data visualization is a specializedand highly valued skill in web development, and the experience gained during this internship provides a strong foundation for future work in this area.

## 9.2 ProfessionalSkillsGained

Beyond technical skills, the internship provided invaluable development of professional competencies that are essential for success in a software engineering career.

- Professional Communication: Working with experienced scientists and engineers at DRDO required clear, concise, and technically precise communication,bothverbalandwritten.Theabilitytoaskeffectivequestions, present technical findings clearly, and discuss complex issues constructively was actively developed throughout the internship.
- Time and Task Management: Managing multiple concurrent tasks — the visualization tool development, bug fixes in existing modules, testing, and documentation—requireddisciplinedtimemanagementandprioritization.The experience of working within the structured environment of a professional research organization was highly instructive in this regard.
- Adaptability:Regularlyencounteringunexpectedchallenges(suchasthedata persistencebugs andthe coordinatetransformation mathematics) andfinding effective solutions under professional constraints developed adaptability and resilience that are critical attributes in any engineering role.
- AttentiontoDetail:Workingonmission-criticalsoftwarewhereerrorscanhave serious operational consequences instilled an exceptionally high standard of attention to detail in coding, testing, and documentation.

# 10. CONCLUSION

## 10.1 Summary

The internship at the Institute for Systems Studies and Analyses (ISSA), DRDO, has been an exceptionally rewarding and professionally formative experience. Over the course of the training programme, meaningful and demonstrable technical contributions were made to the development of a web-based visualization tool and several supporting backend modules, all built on a Java Full Stack technology foundation comprising Java, JSP, HTML, CSS3, JavaScript, and PostgreSQL.

The visualization tool developed during the internship successfully meets all the requirements defined at the outset: it renders dynamic spatial and numerical data in a clear and intuitive sector display using the HTML Canvas API, presents a synchronized informationsummarypanelwithdetailedentityparameters,fetchesandupdatesdata in near-real-time from the backend REST API, and handles error conditions gracefully. The supporting modules were similarly enhanced and corrected, with critical data persistence bugs resolved and comprehensive validation logic implemented.

The technical skills developed during this internship — in Java Full Stack development, JSP, PostgreSQL, Canvas API visualization, RESTful API design, and systematic debugging — represent a substantial and practical enhancement of the academic foundations laid during the B.Tech programme. Equally valuable were the professional skills of communication, time management, adaptability, and attentionto detail that were cultivated within the demanding environment of a premier defense researchorganization.

The internship has reinforced the conviction that the intersection of software engineering and domain-specific applications — whether in defense, healthcare, scientific research, or any other critical field — represents one of the most challenging, rewarding, and impactfulareasinwhichatechnologyprofessionalcancontribute.Theexperienceof

workingatDRDO,howeverbriefly,hassetahighbenchmarkfor professionalstandards and technical rigor that will continue to inspire future work.

## 10.2 FutureScope

Thevisualizationtool and supporting modulesdeveloped during the internship providea solid foundation for several promising future enhancements:

- Real-Time WebSocket Integration: Replacing the current polling-based data refresh mechanism with WebSocket connections would reduce data latency andprovideagenuinelyreal-timedisplayexperience,eliminatingtheperiodic refresh delay that is inherent in the polling approach.
- Mobile-ResponsiveDesign:Enhancingthetool'sCSStofullysupporttablet-sized displays would extend its accessibility to mobile operational contexts.
- User Authentication and Role Management: Implementing a full user authenticationsystemwithrole-basedaccesscontrol(RBAC)wouldallowthe tool to manage different privilege levels for data viewing, configuration management, and administrative functions.

# 11. BIBLIOGRAPHYANDREFERENCES

## Books


- KathySierraandBertBates,'HeadFirstJava'—O'ReillyMedia.
- MartyHallandLarryBrown,'CoreServletsandJavaServerPages'—Prentice Hall.
- JoshuaBloch,'EffectiveJava'—Addison-WesleyProfessional.
- RobertC.Martin,'CleanCode:AHandbookofAgileSoftwareCraftsmanship' —PrenticeHall.
- MartinKleppmann,'DesigningData-IntensiveApplications'—O'ReillyMedia.


## OnlineDocumentation


- OracleJavaSEDocumentation—https://docs.oracle.com/en/java/
- OracleJavaServletSpecification—https://javaee.github.io/servlet-spec/
- JSPDocumentation(JakartaEE)—https://jakarta.ee/specifications/pages/
- PostgreSQL13OfficialDocumentation—https://www.postgresql.org/docs/13/
- MDNWebDocs—HTMLCanvasAPI—https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- MDNWebDocs—JavaScriptReference—https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDNWebDocs—CSSReference—https://developer.mozilla.org/en-US/docs/Web/CSS
- ApacheTomcatDocumentation—https://tomcat.apache.org/tomcat-9.0-doc/
- GitOfficialDocumentation—https://git-scm.com/doc


## OrganizationResources


- ISSA—DRDOOfficialWebsite:https://www.drdo.gov.in


## ToolsUsed

- IntelliJIDEA—JavaIDE—https://www.jetbrains.com/idea/
- PostgreSQL13—https://www.postgresql.org/
- ApacheTomcat9—https://tomcat.apache.org/
- Git—VersionControl—https://git-scm.com/
- pgAdmin4—PostgreSQLGUI—https://www.pgadmin.org/
- MozillaFirefox/GoogleChrome—BrowserTesting